

Polygon Monitoring System Requirements

1. Scope

The system must be **fully modular, scalable, and city-agnostic**.

Any number of cities can be added **without modifying core business logic**.

The architecture is designed for **long-term extensibility, maintainability, and operational reliability**, especially in multi-city deployments.

2. Core Principles

2.1 Configuration over Code

- Cities must **never be hardcoded**
- All city-specific behavior must be defined **exclusively via configuration or data**, including:
 - Geometries
 - Rules
 - Status values
 - Time windows
 - Integrations

2.2 Polygon Zones Are Data

- All geographic zones are **data**, not code artifacts
- Polygon changes must **not require a redeploy**

2.3 Pluggable Rules & Checks

- Checks are implemented as **plugins**
- New rules can be added or replaced **without impacting existing checks**

2.4 Notification Strategy

- **Telegram is the primary notification channel**
- **Twilio SMS is strictly a fallback**
- No parallel notifications

2.5 No Hardcoded Values

- No credentials, time windows, or thresholds in source code
- Everything must be configurable via **config files + environment variables**

- ENV overrides config

3. Existing System (Version 1)

A functional V1 system already exists in the repository:

https://bitbucket.org/ondutyag/uber_polygon_monitoring_system/src/main/

Limitations of V1

- City-specific logic partially embedded in code
- Limited modularity
- Poor separation of:
 - Configuration
 - Data
 - Business logic

🎯 The goal is **not a refactor of V1**, but a **clean evolution to V2**.

4. Target Architecture

```

uber_polygon_system/
├── core/
│   ├── config/                # YAML / JSON loader + ENV overrides
│   (global only)
│   ├── db/                   # DB connections & repositories
│   ├── polygons/             # Polygon loader, validation, registry
│   ├── checks/               # Rule engine & check plugins
│   ├── notifications/        # Telegram (primary) & Twilio (fallback)
│   │   └── templates/        # Message templates (no hardcoded
strings)
│   │   │   ├── telegram/
│   │   │   │   ├── FR/
│   │   │   │   └── DE/
│   │   │   └── sms/
│   │   │       ├── FR/
│   │   │       └── DE/
│   ├── scheduler/            # Periodic execution
│   ├── logging/              # Structured JSON logging
│   └── utils/
│
├── cities/
│   ├── geneva/
│   │   └── config.yaml        # MUST include language.default +
language.supported

```

```

|   |   └─ polygons/
|   |       └─ login_zone/
|   |       └─ waiting_zone/
|   |       └─ delivery_zone/
|   |
|   └─ lausanne/
|       └─ config.yaml    # MUST include language.default +
language.supported
|   |   └─ polygons/
|   |
|   └─ <new_city>/
|       └─ config.yaml    # same mandatory blocks incl. language
|       └─ polygons/
|
└─ cli/
    └─ main.py            # polygons validate|reload, monitor run,
config validate
└─ main.py

```

Acceptance Criteria

✓ Adding a new city requires **only**:

- A city config file
- Polygon CSV files

✗ No core code changes allowed

5. Configuration

5.1 Global Configuration

- YAML or JSON supported
- Environment variables override config values

Configurable items include:

- Database connection (DSN, credentials)
- Scheduler intervals
- Default time windows
- Logging level
- Telegram bot token
- Twilio credentials
- Default language

- Fallback behavior

5.2 City Configuration (Mandatory)

Each city must **fully define its behavior**.

Example: cities/geneva/config.yaml

```
city_code: GE

language:
  default: FR
  supported:
    - FR
    - DE

zones:
  - login_zone
  - waiting_zone
  - delivery_zone

status:
  mapping:
    "ONLINE": online
    "DELIVERING": delivering
    "PAUSED": paused
    "OFFLINE": offline
    "LOGIN_EVENT": login_event

  groups:
    login_candidates: [login_event]
    waiting_candidates: [online, paused]
    delivery_candidates: [online, delivering]

  unknown_policy: log_and_skip # log_and_skip | treat_as_offline |
fail_check

checks:
  login_check:
    enabled: true
    time_window_minutes: 3
    status_group: login_candidates
    target_zone: login_zone
```

```
center_check:
  enabled: true
  time_window_minutes: 10
  status_group: waiting_candidates
  allowed_zones: [waiting_zone]

delivery_check:
  enabled: true
  time_window_minutes: 6
  status_group: delivery_candidates
  target_zone: delivery_zone
```

Status Rule (Critical)

- uber_activity_log_live.status **may differ per city**
- Core logic works **only with canonical statuses**
- Status mapping and grouping are **mandatory per city**

5.3 Language & Localization (Revised – Authoritative)

5.3.1 Language Is Configuration, Not Code

Language handling **must be entirely configuration-driven**.

- ✗ No hardcoded language defaults
- ✗ No language conditionals in business logic
- ✓ All language decisions are resolved by deterministic rules

5.3.2 City Language Is the Absolute Default (Critical)

The city-defined language is always the default language.

There is **no global default language used for business notifications**.

Global language config exists **only as a validation boundary**, not as a runtime fallback.

5.3.3 City Configuration (Mandatory)

Each city must explicitly define exactly one default language and supported languages.

Example: cities/geneva/config.yaml

```
city_code: GE
```

```
language:
  default: FR
  supported:
    - FR
    - DE
```

Rules:

- default **must be included** in supported
- City config **without language block** → **startup failure**
- Adding or changing languages requires **no code changes**
- All notifications **default to the city language**

5.3.4 Telegram Language Resolution (Channel-Specific)

Telegram has **channel-specific language override logic**.

Resolution Order (Telegram ONLY)

1. **users.telegram_language**
 - Used **only if present and valid** (exists in city.supported)
2. **City default language**
3. **No further fallback allowed**

If users.telegram_language:

- Is empty → ignore
- Is unsupported (not FR / DE / etc.) → ignore + log
- Is invalid → ignore + log

Example decision log:

```
{
  "event": "telegram_language_resolved",
  "user_id": "uuid",
  "city": "geneva",
  "resolved_language": "FR",
  "source": "city_default"
}
```

5.3.5 SMS Language Resolution (Strict)

SMS **does NOT** support user-level language overrides.

Resolution Order (SMS)

1. **City default language**
2. **No further fallback**

Rules:

- SMS language is **always city-defined**
- SMS must never use `users.telegram_language`
- Missing SMS template for city language → **hard error + log**
- No silent fallback to other languages

5.3.6 Template Structure (Unchanged, Now Strict)

```
notifications/  
├─ telegram/  
│   ├─ FR/  
│   └─ DE/  
│  
├─ sms/  
│   ├─ FR/  
│   └─ DE/
```

Rules:

- Template selection is **purely data-driven**
- Missing template:
 - Telegram → fallback to city language template
 - SMS → fail notification + log error
- No inline message strings in code

5.3.7 Invalid Language Handling (Mandatory Logging)

All invalid or ignored language inputs **must be logged**.

Example:

```
{  
  "event": "invalid_user_language_ignored",  
  "user_id": "uuid",  
  "channel": "telegram",  
  "provided_language": "ES",  
  "city_default": "FR"  
}
```

6. Polygon Management

6.1 Input

- CSV-based
- One polygon per row
- Format:
 - Column 1: WKT Polygon
 - Column 2: Polygon name

6.2 Processing (Mandatory)

- Parse WKT
- Remove duplicate coordinates
- Auto-close polygons
- Validate geometry
- Convert to Shapely objects
- Cache in memory

6.3 Registry API

```
get_polygon(city, zone_type, polygon_name)
get_all_polygons(city, zone_type)
```

6.4 Fallback Handling

- City-level fallback directories are supported
- All fallback usage must be **explicitly logged**

7. Rule Engine & Checks

7.1 Check Engine

- Scheduler-driven
- Each check implements:
 - query()
 - evaluate()
 - execute()

Requirements

- City-agnostic
- Config-driven
- Reusable

7.2 Required Checks

Login Check

- Detects new logins
- Position $\in$ login_zone
- Writes:
 - login_inside_zone
 - login_outside_zone

Center / Waiting Check

- Applies to online drivers
- Uses average position
- Configurable valid zones
- Writes:
 - pause

Delivery Zone Check

- Applies to all active drivers
- Ensures presence inside delivery_zone
- Writes:
 - out_of_town

7.3 Complex Rules

- Multiple zones
- OR / AND logic
- Defined **only via config**, never in code

8. Notification System

Routing

1. Telegram
2. Twilio SMS (fallback)

User Resolution

Source: users.uuid_id

Required fields:

- telegram_id
- prefix
- phone

Optional:

- language

Telegram

- Sent directly to driver Telegram ID
- Message templates per warning type
- DE / FR language support

SMS

- Used only if Telegram is unavailable
- Phone format: prefix + phone

Error Handling

- Retry on Telegram failure
- Optional SMS fallback after retry
- System errors → Telegram message to IT channel

9. Database

Existing tables:

- uber_activity_log_live
- users
- uber_warnings

No schema changes required.

10. CLI

```
polygons validate
```

```
polygons reload
```

```
monitor run
```

```
monitor run --city=geneva
```

11. Non-Functional Requirements

- Structured JSON logging
- Graceful shutdown
- Healthcheck endpoint
- Polygon caching
- Indexed database queries
- Unit & integration tests
- Secrets via environment variables only

12. Definition of Done

- Geneva and Lausanne fully supported
- New city added without core code changes
- Telegram-first notification flow operational
- SMS fallback verified
- No hardcoded configuration
- Production-ready modular architecture

13. Developer Test Data Requirements

13.1 Purpose

The system **MUST** ship with deterministic polygon fixtures and deterministic database seed inserts so that developers can validate:

1. Polygon membership evaluation (inside/outside)
2. “New login candidate” query logic
3. Warning insertion correctness (uber_warnings)
4. End-to-end check execution correctness (login_check plugin)

All test artifacts **MUST** be reproducible and **MUST** work without modifying core source code or schemas.

13.2 Global Configuration (Tests) — .env Only

For developer tests, **global configuration MUST be provided exclusively via .env**.

- ✗ The test suite **MUST NOT** require a global config YAML/JSON file.
- ✓ All global runtime variables **MUST** be read from environment variables.

13.2.1 Mandatory .env.example

The repository **MUST** contain:

`.env.example`

This file **MUST** define all required environment variables for running tests (values may be dummy).

Minimum required variables:

```
# DB (tests)
DB_HOST=localhost
DB_PORT=3306
DB_USER=onduty
```

```
DB_PASSWORD=changeme
DB_NAME=onduty_bi

# Telegram (tests - mocked or dummy)
TELEGRAM_BOT_TOKEN=dummy
TELEGRAM_IT_CHANNEL_ID=-1000000000000

# Twilio (tests - mocked or dummy)
TWILIO_ACCOUNT_SID=dummy
TWILIO_AUTH_TOKEN=dummy
TWILIO_FROM_NUMBER=+41000000000

# Logging
LOG_LEVEL=INFO
LOG_FORMAT=json

# Test mode
APP_ENV=test
```

13.2.2 ENV Override Rule

All config values **MUST** be resolved from ENV at runtime and in tests.
Any config file values (if present in production) **MUST NOT** override ENV.

13.3 Mandatory Test City Configuration (Authoritative)

The repository **MUST** ship with a deterministic test city that is used to run the developer fixtures.

13.3.1 File Location

```
cities/test_city/config.yaml
```

13.3.2 Mandatory Behavior

The test city **MUST** support login_check with deterministic seed inputs.

Required behaviors:

- **MUST** define city_code: TEST
- **MUST** include the mandatory language block
- **MUST** map LOGIN_EVENT → login_event
- **MUST** define login_candidates = [login_event]
- **MUST** set unknown_policy
- **MUST** enable login_check

- MUST reference login_zone

13.3.3 Authoritative Config Content

The city config MUST be exactly equivalent to:

```
city_code: TEST

language:
  default: FR
  supported:
    - FR
    - DE

zones:
  - login_zone

status:
  mapping:
    "LOGIN_EVENT": login_event
    "ONLINE": online
    "OFFLINE": offline

groups:
  login_candidates: [login_event]

  unknown_policy: log_and_skip    # log_and_skip | treat_as_offline |
fail_check

checks:
  login_check:
    enabled: true
    time_window_minutes: 3
    status_group: login_candidates
    target_zone: login_zone

notifications:
  telegram:
    enabled: true
  sms_fallback:
    enabled: true
```

13.4 Mandatory Simple Polygon Fixture (Authoritative)

The repository MUST include a simple deterministic polygon used for tests to avoid geo ambiguity.

13.4.1 File Location

`cities/test_city/polygons/login_zone/simple_login_zone.csv`

13.4.2 File Format

CSV-based, one polygon per row:

- Column 1: WKT polygon
- Column 2: polygon name

13.4.3 Authoritative File Content

The polygon fixture MUST be exactly:

```
POLYGON((6.60 46.50, 6.64 46.50, 6.64 46.54, 6.60 46.54, 6.60  
46.50)), Login Zone
```

13.5 Mandatory SQL Seed Scripts (Authoritative Inserts)

The repository MUST contain the following seed scripts:

```
tests/fixtures/db_seed/  
├─ seed_users_simple.sql  
├─ seed_login_check_simple.sql  
└─ cleanup_simple.sql
```

These seed scripts MUST:

- be valid MySQL SQL
- run successfully against existing schema
- not require schema modifications
- be deterministic (only NOW() relative timestamps allowed)

13.5.1 seed_users_simple.sql (Authoritative Content)

This script MUST insert three deterministic test users.

✓ Requirements:

- Users MUST include valid prefix and phone
- UUIDs MUST match activity log inserts

- All inserts MUST use UUIDs defined below

The script content MUST be exactly equivalent to:

```
-- =====
-- Seed users for deterministic login_check testing
-- =====

DELETE FROM users
WHERE uuid_id IN ('uuid-in', 'uuid-out', 'uuid-old');

INSERT INTO users (uuid_id, first_name, prefix, phone, telegram_id,
telegram_language)
VALUES
('uuid-in', 'InsideUser', '41', '0790000001', '111111111', 'FR'),
('uuid-out', 'OutsideUser', '41', '0790000002', '222222222', 'FR'),
('uuid-old', 'OldUser', '41', '0790000003', '333333333', 'FR');
```

13.5.2 seed_login_check_simple.sql (Authoritative Content)

This script MUST seed uber_activity_log_live such that:

- ✓ uuid-in is a new login inside polygon
- ✓ uuid-out is a new login outside polygon
- ✗ uuid-old is NOT a new login (excluded due to older record inside last 2 hours)

13.5.2.1 Deterministic Points

The polygon used MUST be the simple square polygon defined in 13.4.

Test points MUST be:

- ✓ INSIDE point: longitude **6.6200**, latitude **46.5200**
- ✗ OUTSIDE point: longitude **6.7000**, latitude **46.5200**

13.5.2.2 City Code Alignment (Mandatory)

All seed records MUST use:

- location = 'TEST'

This MUST match cities/test_city/config.yaml (city_code: TEST).

13.5.2.3 Authoritative Seed Script

The script content MUST be exactly equivalent to:

```

-- =====
-- Seed activity log for deterministic login_check test
-- =====

DELETE FROM uber_activity_log_live
WHERE uuid IN ('uuid-in', 'uuid-out', 'uuid-old');

-- -----
-- uuid-in: NEW login inside polygon
-- Should appear in login_check query
-- Expected warning: login_inside_zone
-- -----
INSERT INTO uber_activity_log_live (uuid, latitude, longitude,
location, status, ts)
VALUES
('uuid-in', 46.5200, 6.6200, 'TEST', 'LOGIN_EVENT', NOW() - INTERVAL
1 MINUTE);

-- -----
-- uuid-out: NEW login outside polygon
-- Should appear in login_check query
-- Expected warning: login_outside_zone
-- -----
INSERT INTO uber_activity_log_live (uuid, latitude, longitude,
location, status, ts)
VALUES
('uuid-out', 46.5200, 6.7000, 'TEST', 'LOGIN_EVENT', NOW() -
INTERVAL 1 MINUTE);

-- -----
-- uuid-old: NOT a new login (must be excluded)
-- Must have an older record within the last 2 hours
-- and a fresh record within last 3 minutes.
-- Expected: ignored (no warning)
-- -----
INSERT INTO uber_activity_log_live (uuid, latitude, longitude,
location, status, ts)
VALUES
('uuid-old', 46.5200, 6.6200, 'TEST', 'ONLINE',      NOW() -
INTERVAL 30 MINUTE),

```



```
('uuid-old', 46.5200, 6.6200, 'TEST', 'LOGIN_EVENT', NOW() -  
INTERVAL 1 MINUTE);
```

13.5.3 cleanup_simple.sql (Authoritative Content)

This script MUST remove the seeded test records and any warnings created by running login_check.

The script content MUST be exactly equivalent to:

```
-- =====  
-- Cleanup deterministic seed data for login_check  
-- =====  
  
DELETE FROM uber_activity_log_live  
WHERE uuid IN ('uuid-in', 'uuid-out', 'uuid-old');  
  
DELETE FROM uber_warnings  
WHERE driver_id IN ('uuid-in', 'uuid-out', 'uuid-old');  
  
DELETE FROM users  
WHERE uuid_id IN ('uuid-in', 'uuid-out', 'uuid-old');
```

13.6 Seed Test Expected Results (Mandatory)

After running seed_users_simple.sql, seed_login_check_simple.sql, and executing the login_check plugin:

13.6.1 Expected Candidate Selection (Query Output)

The login candidate query MUST return:

- ✓ uuid-in
- ✓ uuid-out

The login candidate query MUST NOT return:

- ✗ uuid-old

13.6.2 Expected Warnings Inserted

The system MUST insert the following uber_warnings.note values:

UUID	Expected note
uuid-in	login_inside_zone
uuid-out	login_outside_zone
uuid-old	no warning inserted

13.7 Mandatory Verification Queries (Must Be Documented)

13.7.1 Login Candidate Query

The repository MUST include this query in test documentation:

```
SELECT DISTINCT t1.uuid
FROM uber_activity_log_live t1
WHERE t1.ts BETWEEN NOW() - INTERVAL 3 MINUTE AND NOW()
AND t1.location = 'TEST'
AND NOT EXISTS (
  SELECT 1
  FROM uber_activity_log_live t2
  WHERE t2.uuid = t1.uuid
  AND t2.location = 'TEST'
  AND t2.ts BETWEEN NOW() - INTERVAL 2 HOUR AND NOW() - INTERVAL 3
MINUTE
);
```

Expected output: uuid-in, uuid-out

13.7.2 Warnings Verification

The repository MUST include this query:

```
SELECT driver_id, note
FROM uber_warnings
WHERE driver_id IN ('uuid-in', 'uuid-out', 'uuid-old');
```

13.8 CI Requirement (Mandatory)

CI MUST run the following steps:

1. Load .env.example or CI env variables
2. Spin up MySQL (docker/testcontainers)
3. Apply seed scripts
4. Execute login_check plugin evaluation
5. Verify warnings match expected results
6. Cleanup seed records

CI MUST fail if any expected results are missing or incorrect.

13.9 Adding New City Testing Requirement

Any new city MUST add:

- cities/<new_city>/config.yaml
- polygon CSVs in the required directory structure
- required templates
- deterministic seed SQL scripts
- expected results file

 No core code changes allowed.